

Exceções em Flutter

O tratamento de exceções é uma parte fundamental do desenvolvimento de aplicações robustas. Em Flutter, a detecção e o gerenciamento de erros permitem criar experiências de usuário mais suaves e confiáveis. Este texto aborda os principais tipos de exceções em Flutter, como tratá-las adequadamente e práticas recomendadas para evitar falhas inesperadas.



Fonte: <https://pixabay.com/pt/vectors/codificação-computador-1294361/>

O que são Exceções?

Exceções são eventos inesperados que ocorrem durante a execução de um programa, resultando em interrupções ou comportamento inesperado. Em Flutter, elas podem ser causadas por diversos fatores, como falhas de rede, entradas inválidas ou acessos a recursos inexistentes.

Tipos de Exceções Comuns em Flutter

FormatException:

Ocorre quando o formato dos dados fornecidos não é válido.

Exemplo: Ao tentar converter uma string inválida em um número.

TimeoutException:

Lançada quando uma operação leva mais tempo do que o esperado.

SocketException:

Relacionada a problemas de conexão de rede.

AssertionError:

Aparece quando uma afirmação (assert) falha durante a execução em debug.

FlutterError:

Específica do Flutter, ocorre em situações como erros na árvore de widgets.

Tratamento de Exceções

O Flutter oferece ferramentas robustas para tratar exceções e minimizar o impacto de erros no app. Alguns métodos comuns incluem:

Try-Catch:

Usado para capturar e lidar com exceções específicas ou genéricas.

```
try {  
  int result = int.parse('abc');  
} catch (e) {  
  print('Erro: $e');  
}
```

Finally:

Adiciona um bloco para executar código independentemente de a exceção ter ocorrido ou não.

```
try {  
  // Operação  
} catch (e) {  
  // Tratamento  
} finally {  
  print('Execução concluída');  
}
```

Async e Await com Tratamento:

Exceções em operações assíncronas são tratadas usando os mesmos blocos try-catch.

ErrorWidget.builder:

Personaliza a interface exibida em caso de erro na árvore de widgets.

```
ErrorWidget.builder = (FlutterErrorDetails details) {  
  return Center(child: Text('Ocorreu um erro!'));  
};
```

Boas Práticas no Tratamento de Exceções

- **Evite Silenciar Erros:** Sempre registre ou notifique erros inesperados para investigação posterior.
- **Use Logs:** Ferramentas como Firebase Crashlytics ajudam a monitorar exceções em produção.
- **Feedback ao Usuário:** Informe o usuário quando um erro ocorrer e, se possível, ofereça soluções.
- **Validações Antecipadas:** Prevenir erros com validações adequadas antes de executar operações.

GitHub da Disciplina

Você pode acessar o repositório da disciplina no GitHub a partir do seguinte link:

<https://github.com/FaculdadeDescomplica/Pratica-Integradora-Desenvolvimento-de-Apps>. Esse espaço é o seu portal para mergulhar fundo no universo da

aprendizagem interativa. Nele, você encontrará todos os códigos, além dos links para os arquivos e dados.

Conteúdo Bônus

Confira mais detalhes sobre tratamento de exceções em Flutter neste guia completo:

Título: Handling errors in Flutter

Plataforma: Flutter

Referências Bibliográficas

BIESSEK, Alessandro. Flutter for Beginners. 1. ed. Birmingham: Packt Publishing, 2020.

SEJOURNÉ, Philippe. Flutter: Aplicações Nativas Multiplataforma com Flutter e Dart. 1. ed. Rio de Janeiro: Alta Books, 2020.

ZAMMETTI, Frank. Practical Flutter: Improve your Mobile Development with Google's Latest Open-Source Framework. 1. ed. Berkeley: Apress, 2019.